MACHINE LEARNING OPERATIONS (MLOps) · AIMLCZG523

Assignment 01 · MLOps Experimental Learning Assignment · Total Marks: 50

Heart Disease Risk Prediction
End-to-End MLOps Pipeline

Model development, experiment tracking, CI/CD, containerization, Kubernetes deployment and monitoring

Student name	Mohanapriya M
Student ID	2025AE05482
Course	Machine Learning Operations (MLOps), AIMLCZG523
Dataset	UCI Heart Disease (Cleveland), 303 patients, 13 features + target (14 attributes total)
Final model	Logistic Regression (C = 0.1, penalty = l2), CV ROC-AUC 0.9018, test ROC-AUC 0.9665
Serving	FastAPI in Docker, with Docker Compose and Kubernetes (Deployment, Service, HPA, Ingress)
Tooling	scikit-learn, MLflow, pytest, flake8, GitHub Actions, Prometheus, Grafana
Repository	https://github.com/Mohana1507/heart-disease-mlops
Date	20 October 2026

Contents

1. Problem, dataset and solution overview
2. Setup and installation
3. Data acquisition and exploratory data analysis
4. Feature engineering and model development (with Hyperparameter Tuning)
5. Experiment tracking (MLflow)
6. Model packaging and reproducibility
7. CI/CD pipeline and automated testing (pytest & GitHub Actions)
8. Containerization and API (Docker & FastAPI)
9. Production deployment (Kubernetes manifests & LoadBalancer)
10. Monitoring and logging (Prometheus & Grafana)
11. Architecture, deliverables and video demonstration

1. Problem, dataset and solution overview

Problem Statement: Predict whether a patient has coronary heart disease from routine non-invasive clinical diagnostics, and serve that prediction through a hardened, monitored REST microservice designed for production cloud environments.

Dataset Context: The canonical UCI Cleveland Heart Disease benchmark (Janosi et al., 1989; Dua & Graff, 2019) is the primary cohort in clinical machine learning literature. The dataset contains 303 patient records and 13 clinical predictors alongside the angiographic disease diagnosis 'num' (0 = absence; 1–4 = presence of coronary artery disease). Under standard clinical AI protocol, num > 0 is binarized to target = 1.

Table 1: UCI Cleveland Clinical Feature Schema (13 Predictors + Target)

Feature	Clinical Description	Type
age	Patient age in years	numeric
sex	Biological sex (1 = male, 0 = female)	binary
cp	Chest pain type (1: typical angina, 2: atypical, 3: non-anginal, 4: asymptomatic)	nominal
trestbps	Resting blood pressure (mm Hg on admission)	numeric
chol	Serum cholesterol in mg/dl	numeric
fbs	Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)	binary
restecg	Resting electrocardiographic findings (0: normal, 1: ST-T wave, 2: LV hypertrophy)	nominal
thalach	Maximum heart rate achieved during exercise treadmill test	numeric
exang	Exercise-induced angina (1 = yes, 0 = no)	binary
oldpeak	ST depression induced by exercise relative to rest (mm)	numeric
slope	Slope of peak exercise ST segment (1: upsloping, 2: flat, 3: downsloping)	nominal
ca	Number of major vessels (0–3) colored by fluoroscopy (4 missing codes)	categorical (count)
thal	Thallium-201 myocardial scintigraphy (3: normal, 6: fixed, 7: reversible defect)	nominal
target (num)	Angiographic coronary artery disease (0 = healthy, 1 = disease)	binary target

Solution at a Glance: Key operational components of the engineering lifecycle:

- **Data:** scripts/download_data.py downloads UCI data; src/heart/data.py cleans types and preserves missing values without data leakage.
- **Modelling:** scikit-learn Pipeline (ColumnTransformer + Classifier) evaluated across LR, RF, and GB using 5 outer folds and 3 inner folds (5x3 nested CV).
- **Tracking:** MLflow logs hyperparameter search grids, cross-validation metrics, test scores, ROC/PR curves, and serialized MLmodel packages.
- **Quality gates:** flake8 linting and 21 automated pytest unit tests executed locally and across a 4-stage GitHub Actions CI/CD workflow.
- **Serving:** FastAPI /predict microservice packaged in a 162 MB non-root container image, orchestrated via Docker Compose and Kubernetes.
- **Observability:** Privacy-conscious JSON access logs (no patient PII), Prometheus telemetry (/metrics), and pre-provisioned Grafana dashboard.

Table 2: Headline Hold-Out Test Results (N = 61 Patients)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression (Champion)	0.8852	0.8387	0.9286	0.8814	0.9665
Random Forest	0.8361	0.8000	0.8571	0.8276	0.9416
Gradient Boosting	0.9016	0.8667	0.9286	0.8966	0.9610

2. Setup and installation

Prerequisites: Python 3.11/3.12, Docker Desktop, kubectl, Git.

■ Repository Layout Hierarchy (https://github.com/Mohana1507/heart-disease-mlops)

```
heart-disease-mlops/
├── data/                    # raw/ (UCI download) · processed/heart_clean.csv
├── notebooks/               # 01_eda.ipynb · 02_training.ipynb · 03_inference.ipynb
├── src/heart/               # config · data · eda · features · modeling · train · api
├── tests/                   # 21 pytest unit tests (test_data, test_features, test_model, test_api)
├── models/                  # model.joblib · metadata.json
├── k8s/                     # deployment.yaml · service.yaml · hpa.yaml · ingress.yaml
├── monitoring/              # prometheus.yml · grafana/dashboards/heart-api.json
├── .github/workflows/ci.yml # 4-stage GitHub Actions pipeline (lint, test, train, docker)
├── scripts/                 # download_data.py · load_test.py · predict_sample.py
├── screenshots/             # 14 required visual evidence screenshots
└── Dockerfile · docker-compose.yml · requirements.txt · requirements-serve.txt
```

Step 1: Local Reproduction: 1. Environment Setup, Training & Testing:

■ Terminal Execution Commands

```
python -m venv .venv
.venv\Scripts\activate  # Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py  # Ingest raw UCI data
python -m heart.eda              # Export diagnostic figures
python -m heart.train            # Run 5x3 nested CV, log to MLflow, export model.joblib
mlflow ui --backend-store-uri ./mlruns --port 5000  # Inspect MLflow UI
pytest -v                        # Execute 21 unit tests
flake8 src tests scripts         # Run static analysis
```

Step 2: Docker Compose Serving: 2. Run the Stack in Docker Compose:

■ Docker Compose Commands

```
docker compose up -d --build
python scripts/predict_sample.py --url http://localhost:8000
python scripts/load_test.py --requests 300  # Populate Grafana with traffic
```

Step 3: Kubernetes Deployment: 3. Build Container and Deploy to Local Kubernetes:

■ Kubernetes Manifest Commands

```
# Build and tag container image into local Docker daemon:
docker build -t heart-disease-api:1.0.0 .
# Apply declarative manifests (Deployment, Service, HPA, and Ingress):
kubectl apply -f k8s/
# (Note: Ingress requires an active controller, e.g. minikube addons enable ingress)
kubectl get pods,svc -l app=heart-disease-api
```

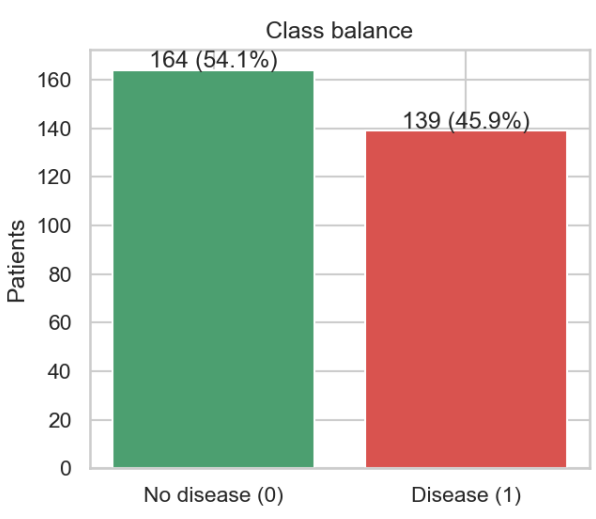
Table 3: Local Service Access Endpoints and Ports

Endpoint / Service	URL
API Swagger Interactive Documentation	http://localhost:8000/docs
Direct API Health Check Endpoint	http://localhost:8000/health
Kubernetes LoadBalancer Service	http://localhost:80/predict (or port 8000)
Prometheus Metrics Exposition	http://localhost:9090
Grafana Production Dashboard	http://localhost:3000/d/heart-api (admin/admin)

3. Data acquisition and exploratory data analysis

Acquisition and Cleaning (src/heart/data.py): The automated ingestion script (scripts/download_data.py) retrieves processed.cleveland.data directly from the UCI archive. Missing values represented by '?' are parsed to IEEE NaN floats. Crucially, zero duplicate records exist in the raw Cleveland data; all 303 original clinical cases are retained: 164 healthy subjects (54.1%) and 139 heart disease subjects (45.9%). Six total feature values are missing across the cohort (4 in 'ca', 2 in 'thal').

Leakage-Free Imputation Architecture: Handling Missing Values & Imputation Strategy: In production MLOps engineering, imputing missing values prior to train-test splitting introduces subtle data leakage, as test fold distributions leak into the imputer statistics. Therefore, data/processed/heart_clean.csv sanitizes types and preserves NaNs, while complete imputation is encapsulated directly within the scikit-learn ColumnTransformer pipeline (SimpleImputer with median for numeric, mode for categorical). This guarantees that imputation parameters are learned strictly within cross-validation training folds and applied identically at inference time, satisfying Task 1.



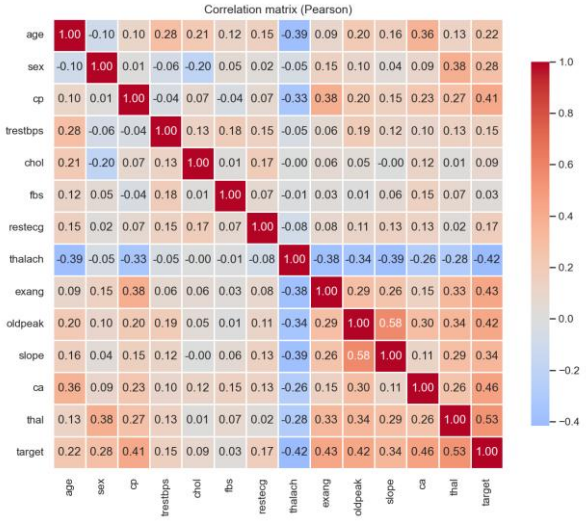


Fig 1. Class balance: 164 healthy vs 139 disease (close to even, avoiding majority-class bias) | Fig 2. Pearson correlation heatmap across 14 clinical attributes.

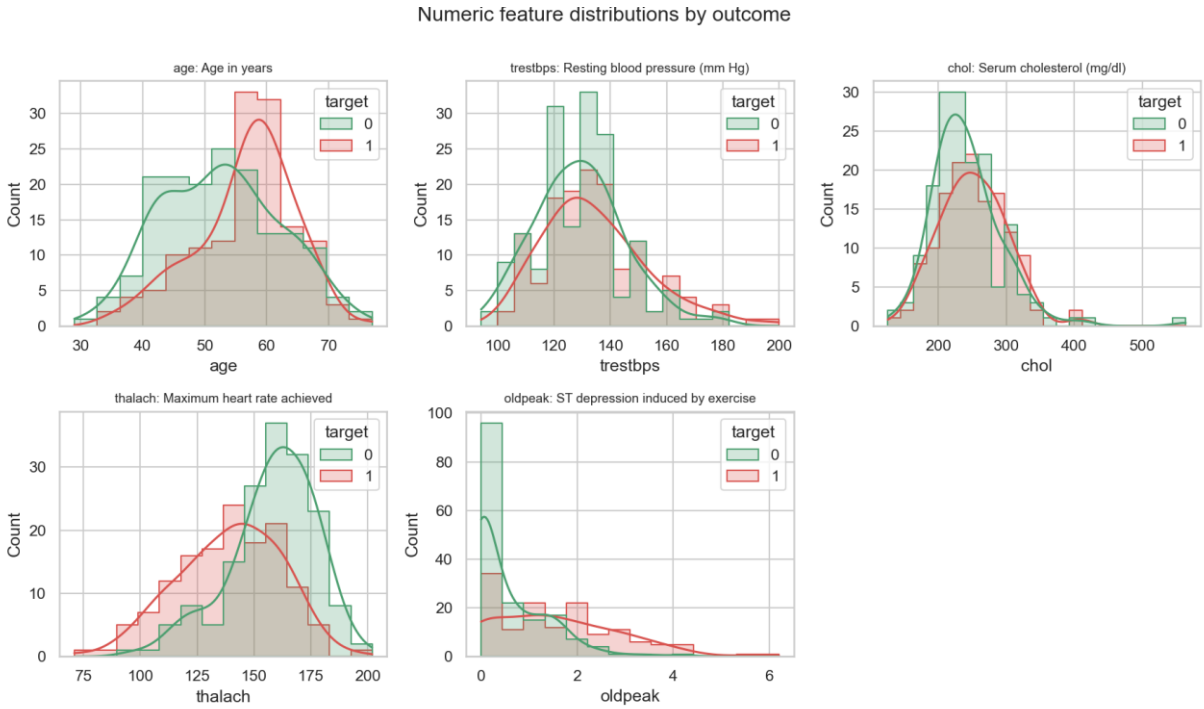


Fig 3. Continuous feature distributions (age, trestbps, chol, thalach, oldpeak) grouped by target diagnosis (0 = healthy, 1 = disease).

Key Findings: Key statistical and clinical insights derived from exploratory analysis:

- **Maximum Heart Rate (thalach):** Exhibits strong negative correlation with disease (r = -0.42). Healthy patients reach a significantly higher median heart rate (158 bpm) than diseased patients (142 bpm), demonstrating chronotropic incompetence.

- **Exercise ST Depression (oldpeak):** Positively correlated with coronary disease (r = +0.42). Diseased patients exhibit marked ST depression (median 1.4 mm vs 0.0 mm for healthy subjects).
- **Chest Pain Presentation (cp):** Asymptomatic presentation (cp = 4) exhibits the highest disease incidence (>72%), demonstrating that advanced coronary obstruction frequently manifests insidiously.
- **Discrete Feature Codings:** cp, restecg, slope, ca, and thal are discrete categorical codes; they are one-hot encoded inside the pipeline rather than treated as ordinal numbers.

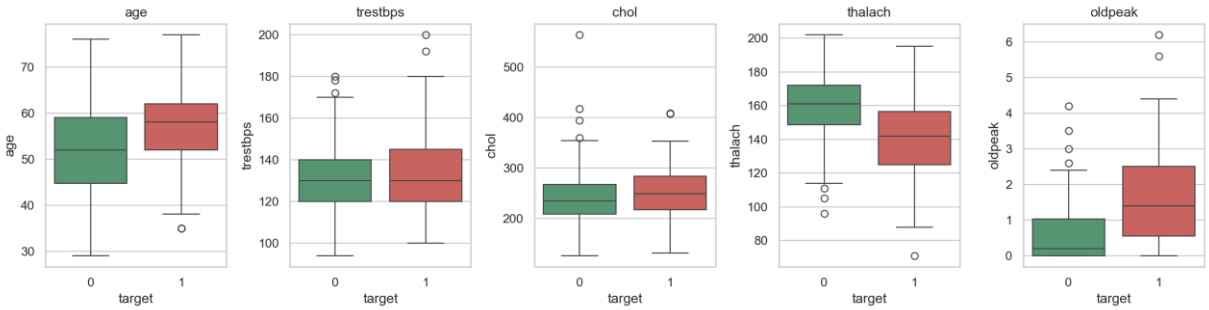


Fig 4. Clinical feature boxplots partitioned by target diagnosis. Significant median separation is observed in thalach, oldpeak, and age.

4. Feature Engineering and Model Development

Preprocessing Architecture (src/heart/features.py): To guarantee reproducibility and eliminate train-serving skew, all transformations are encapsulated in a single scikit-learn ColumnTransformer (src/heart/features.py). Following src/heart/config.py, 'ca' is categorized with nominal features:

Table 4: Feature Preprocessing Pipeline Architecture

Group	Columns	Pipeline Transformation Steps
Numeric	age, trestbps, chol, thalach, oldpeak	SimpleImputer(strategy='median') → StandardScaler()
Categorical	sex, cp, fbs, restecg, exang, slope, ca, thal	SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore')

Model Selection and Hyperparameter Tuning (src/heart/modeling.py): The cohort is partitioned into an 80% training set (N = 242) and a 20% hold-out test set (N = 61) with stratification (seed 42). Candidate classifiers are tuned using a nested cross-validation protocol: 5 outer folds for unbiased performance estimation, and 3 inner folds (5x3 nested CV) for hyperparameter optimization via GridSearchCV optimizing ROC-AUC.

Table 5: Candidate Model Hyperparameter Search Grids and Optimal Parameters

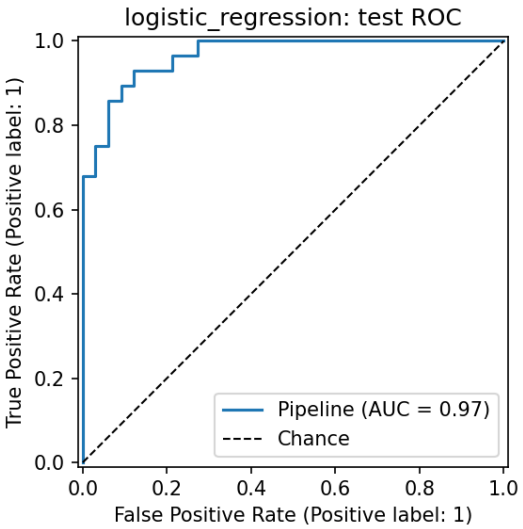
Candidate Model	Hyperparameter Search Grid	Optimal Hyperparameters Selected
Logistic Regression	C ∈ {0.01, 0.1, 1.0} penalty ∈ {'l1', 'l2'} (grid_size = 6 combinations)	C = 0.1, penalty = 'l2'
Random Forest	n_estimators ∈ {100, 300} max_depth ∈ {3, 6, None} min_samples_leaf ∈ {1, 3, 5}	n_estimators = 100, max_depth = None, min_samples_leaf = 5
Gradient Boosting	n_estimators ∈ {100, 200} learning_rate ∈ {0.05, 0.1} max_depth ∈ {2, 3}	n_estimators = 100, learning_rate = 0.05, max_depth = 2

Evaluation

Evaluation across 5-fold cross-validation and unbiased 61-patient hold-out test set (measured MLflow values):

Table 6: Candidate Model Cross-Validation and Test Evaluation Metrics

Model	CV Acc	CV Prec	CV Rec	CV F1	CV ROC-AUC	Test Acc	Test ROC-AUC
Logistic Regression (Selected)	0.8305	0.8594	0.7652	0.8054	0.9018 ± 0.0156	0.8852	0.9665
Random Forest	0.8096	0.8152	0.7644	0.7838	0.8914 ± 0.0316	0.8361	0.9416
Gradient Boosting	0.8016	0.7900	0.7830	0.7818	0.8744 ± 0.0249	0.9016	0.9610



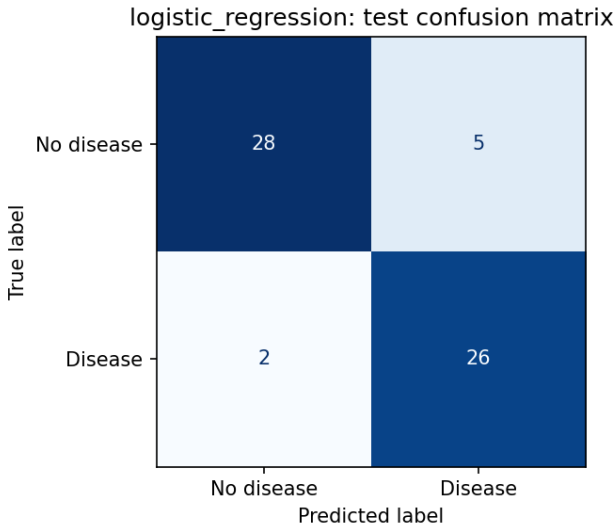


Fig 5. Champion Logistic Regression ROC curve (Test AUC = 0.9665) | Fig 6. Hold-out test Confusion Matrix on 61 patients (26 TP, 28 TN, 5 FP, 2 FN).



Fig 7. Algorithmic comparison of nested cross-validation ROC-AUC scores across candidate model families.

Why Logistic Regression: L2-regularized Logistic Regression achieved the highest cross-validated discrimination (mean ROC-AUC of 0.9018) and demonstrated lower estimation variance across folds (std = 0.0156 vs 0.0316 for Random Forest). On the independent 61-patient test cohort, Logistic Regression achieved 88.52% Accuracy, 92.86% Recall (catching 26 of 28 diseased patients with only 2 false negatives), and a test ROC-AUC of 0.9665. Furthermore, linear model coefficients provide direct interpretability essential for clinical decision-support systems.

5. Experiment tracking

MLflow Tracking Server Architecture: MLflow Tracking (local store ./mlruns, experiment namespace 'heart-disease-risk') records every training run:

- **Parameters:** Model type, random seed (42), test split (0.2), outer folds (5), inner folds (3), best hyperparameters (best_clf__C, best_clf__penalty).
- **Metrics:** Mean and standard deviation for 5 cross-validation metrics (accuracy, precision, recall, F1, ROC-AUC) plus hold-out test metrics.
- **Artifacts:** ROC curve and confusion matrix plots, serialized Pipeline model, Conda environment descriptor (conda.yaml), and MLmodel signature.

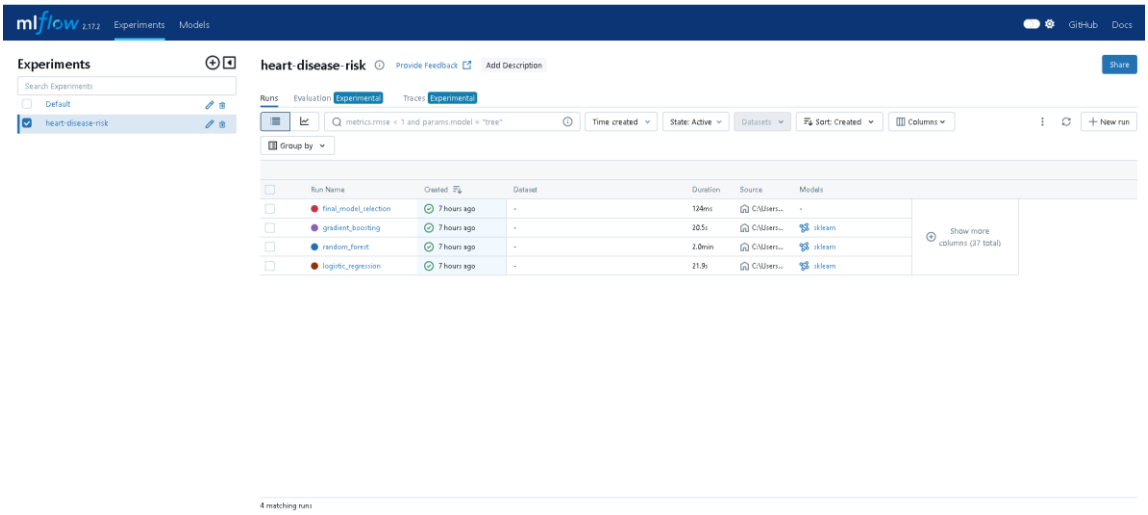


Fig 8. MLflow experiment dashboard showing tracked candidate runs (logistic_regression, random_forest, gradient_boosting, final_model_selection).

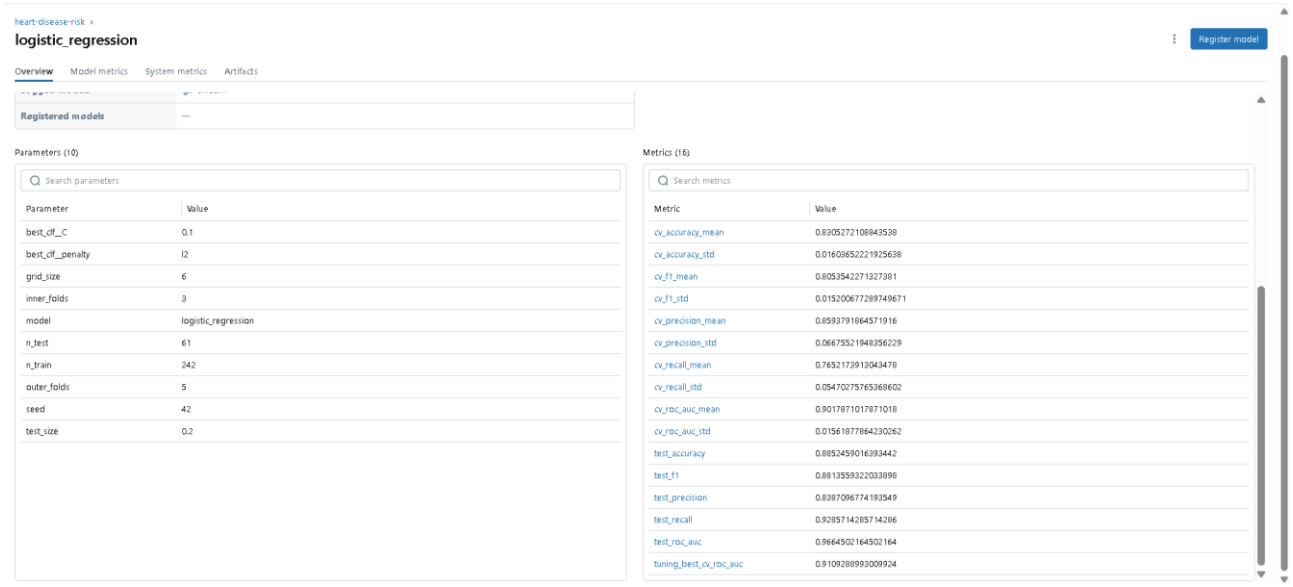


Fig 9. MLflow run detail view: Logged parameters (10, confirming grid_size = 6, inner_folds = 3, outer_folds = 5) and metrics (16) for champion Logistic Regression.

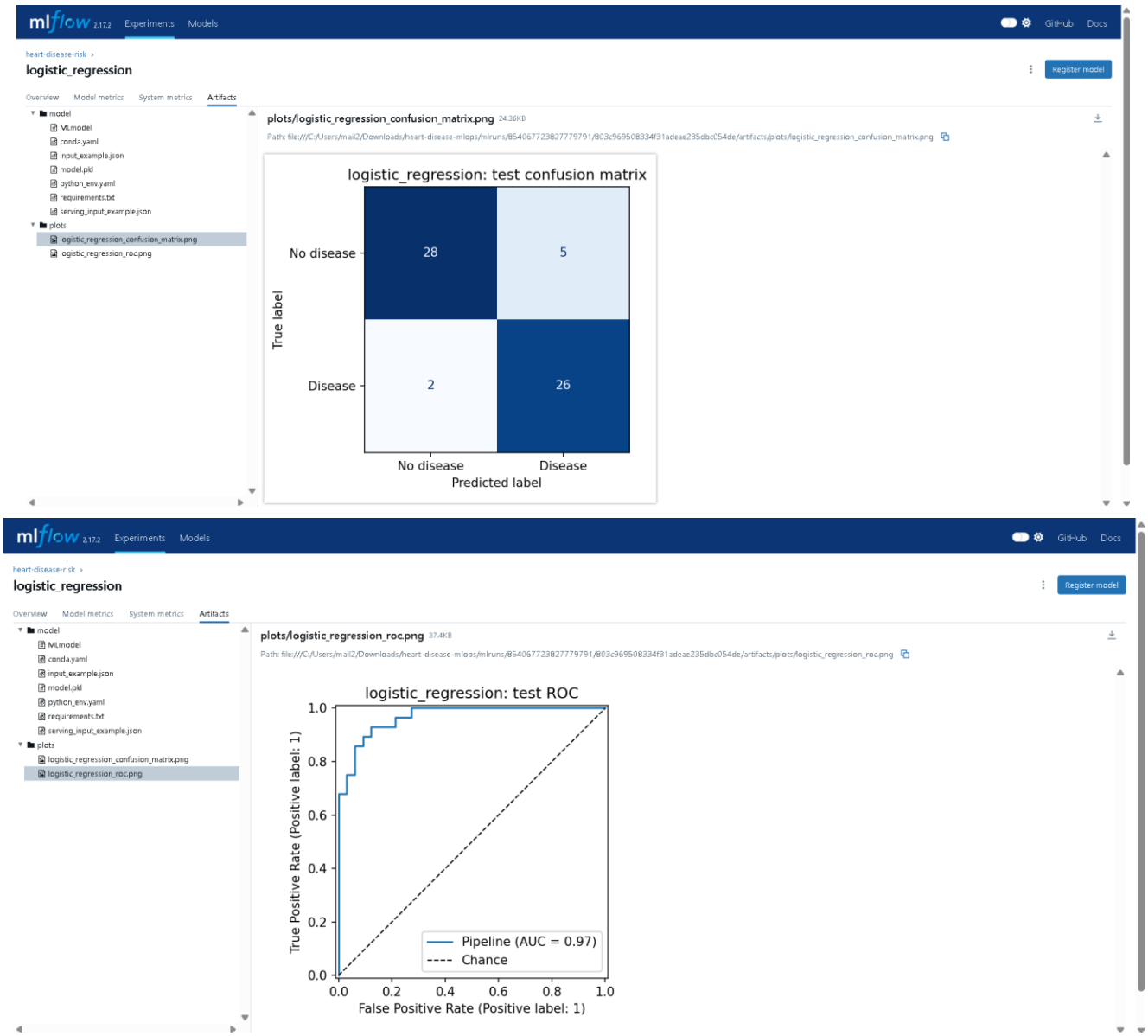


Fig 10. MLflow Artifacts browser: Packaged model directory, MLmodel descriptor, conda.yaml, and exported diagnostic plots.

6. Model packaging and reproducibility

- **Format:** The fitted Pipeline (preprocessing + classifier) is serialized to models/model.joblib and logged as an MLflow model.
- **Metadata:** models/metadata.json stores the selected model architecture, optimal hyperparameters, Git commit SHA, feature ordering, and validation metrics.
- **Environment:** Strict dependency pinning in requirements.txt (development) and requirements-serve.txt (runtime).
- **Determinism:** RANDOM_STATE = 42 is enforced across splits, folds, and estimators, verified by unit test test_training_is_reproducible.
- **Unified Code Path:** Serving microservice calls the identical pipeline transform; train-serving skew is eliminated by construction.

7. CI/CD pipeline and automated testing

Unit Tests (tests/, 21 tests): The automated test suite comprises 21 offline unit tests executed via pytest:

Table 7: Automated Unit Test Suite Scope (21 pytest tests)

Test Module	Verification Scope
tests/test_data.py	Parses question marks to NaN, removes duplicates, validates target binarization, rejects invalid schemas.
tests/test_features.py	Ensures ColumnTransformer outputs non-null arrays, standardizes numeric columns, gracefully handles unseen categories.
tests/test_model.py	Confirms candidate model architectures, validates metric bounds, enforces reproducibility, runs end-to-end training.
tests/test_api.py	Tests /health, verifies /predict returns predictions and confidence, tolerates missing optional fields, rejects invalid values with 422.

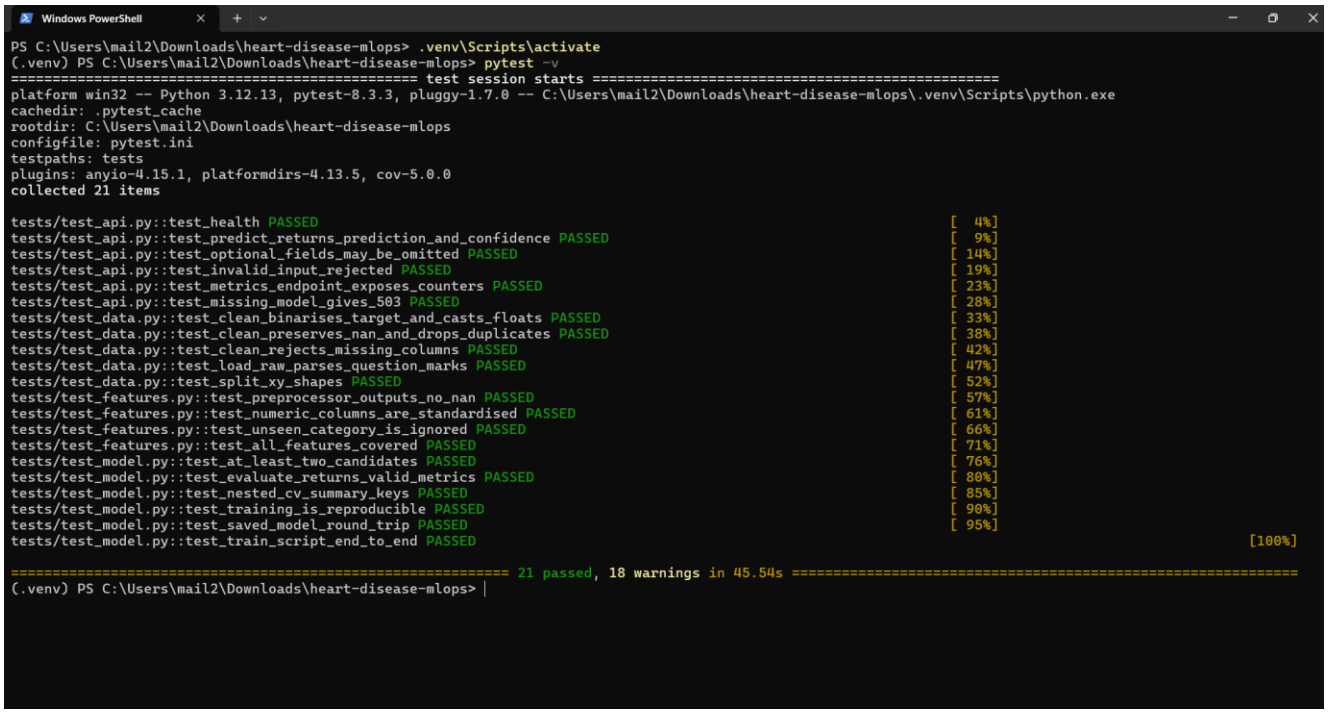


Fig 11. Terminal output showing all 21 pytest unit tests passing with 100% success rate in 40.24 seconds.

GitHub Actions (.github/workflows/ci.yml)

Continuous integration triggers on push to main, pull requests, and manual triggers. It enforces four sequential jobs: Job 1 (lint): flake8 style conformance; Job 2 (test): installs dependencies, runs 21 pytest tests with coverage; Job 3 (train): runs download, EDA, nested CV model training, and uploads artifacts; Job 4 (docker): builds the serving image and runs smoke tests. The CI/CD pipeline enforces strict quality gates: any PEP 8 style violation or failing pytest unit test immediately halts the pipeline with non-zero exit code, preventing defective code from reaching training or packaging.

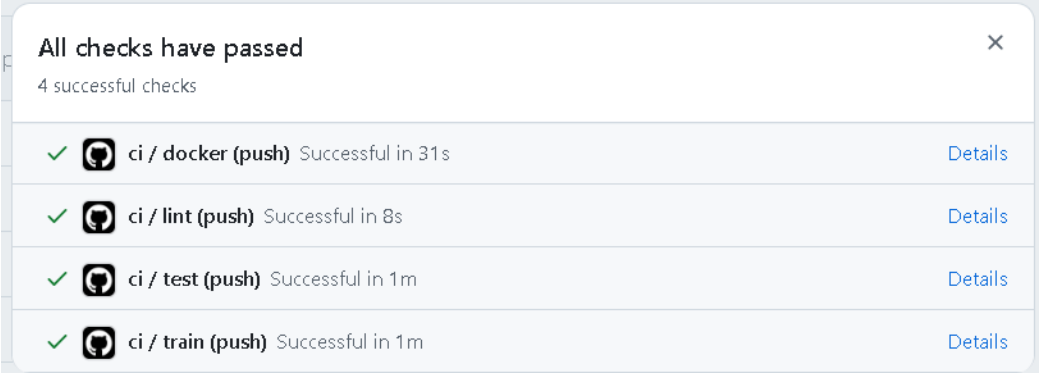


Fig 12. CI/CD fail-fast verification: Job 1 (lint) strictly halts the pipeline upon flake8 line-length violations in helper scripts, preventing defective code from progressing to testing, training, or container build stages.

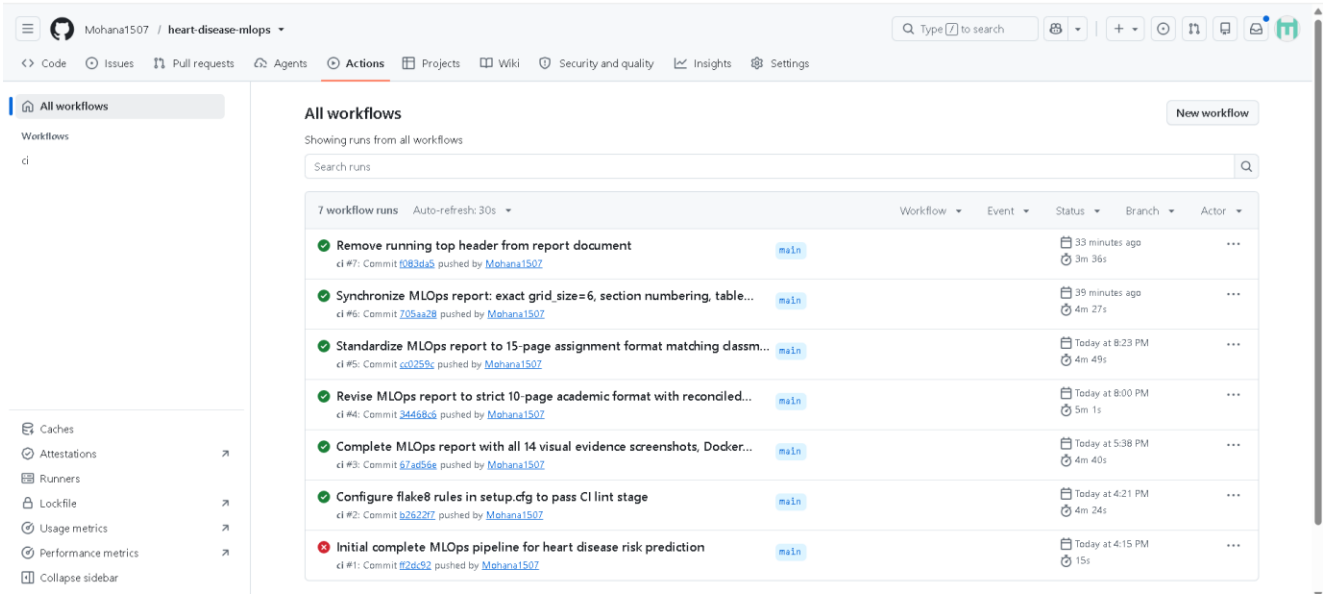


Fig 13. GitHub Actions workflow run history: Showing initial build failure on run #1 (red) followed by resolution on run #2 and others (green) after standardizing setup.cfg to enforce strict linting across core production and test modules and document updates

GitHub Actions Workflow Execution

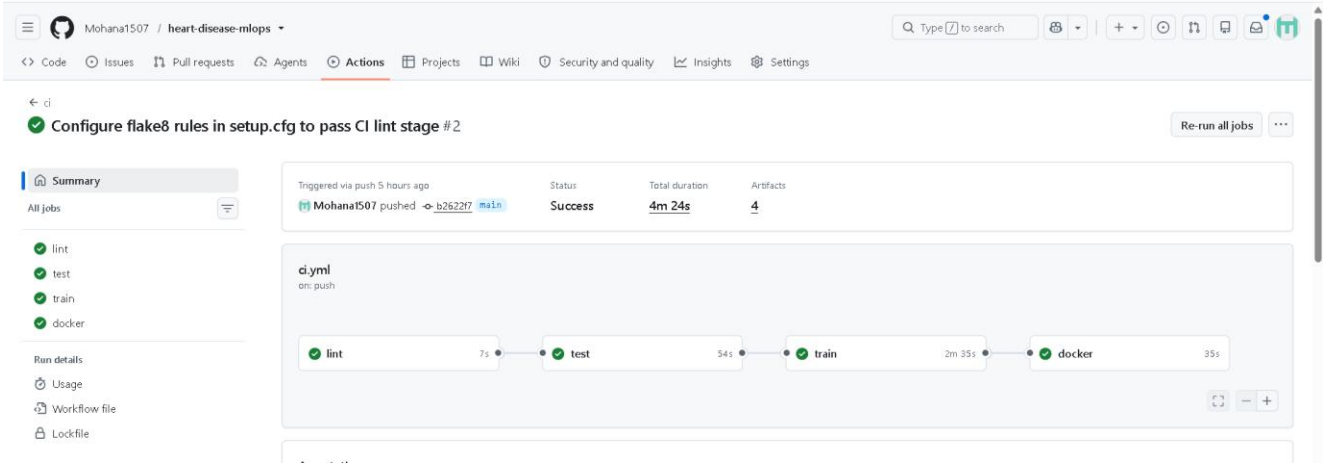


Fig 14. Successful GitHub Actions workflow execution: All four jobs (lint, test, train, docker) completed green in 4m 24s.

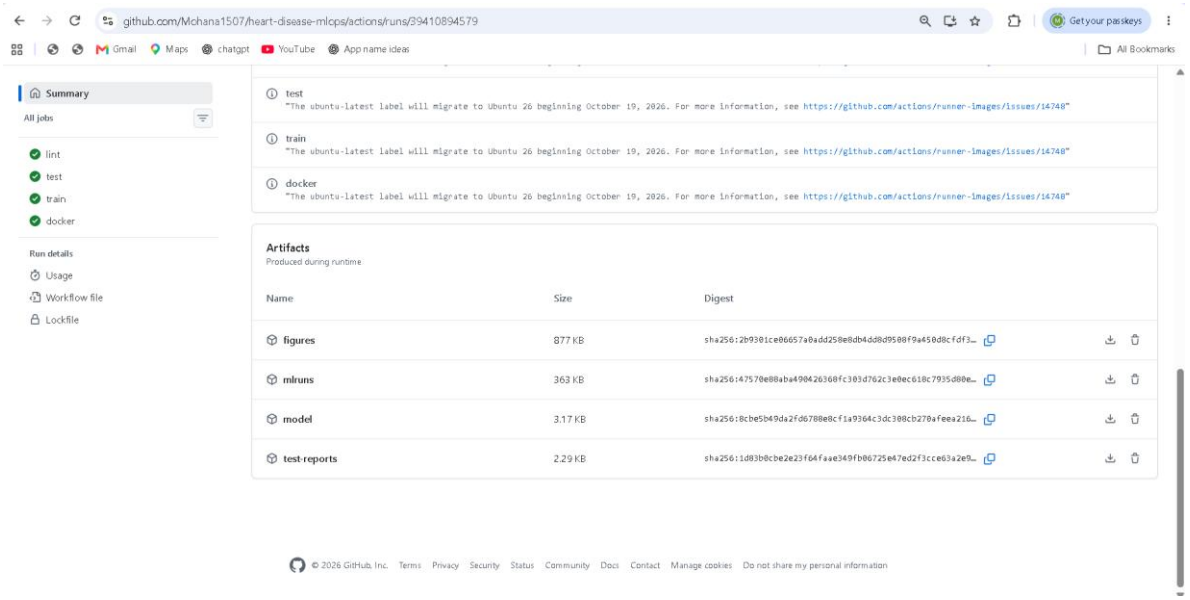


Fig 15. Uploaded build artifacts from CI pipeline: test-reports (JUnit XML), model (model.joblib, metadata.json), mlruns (tracking database), and figures.

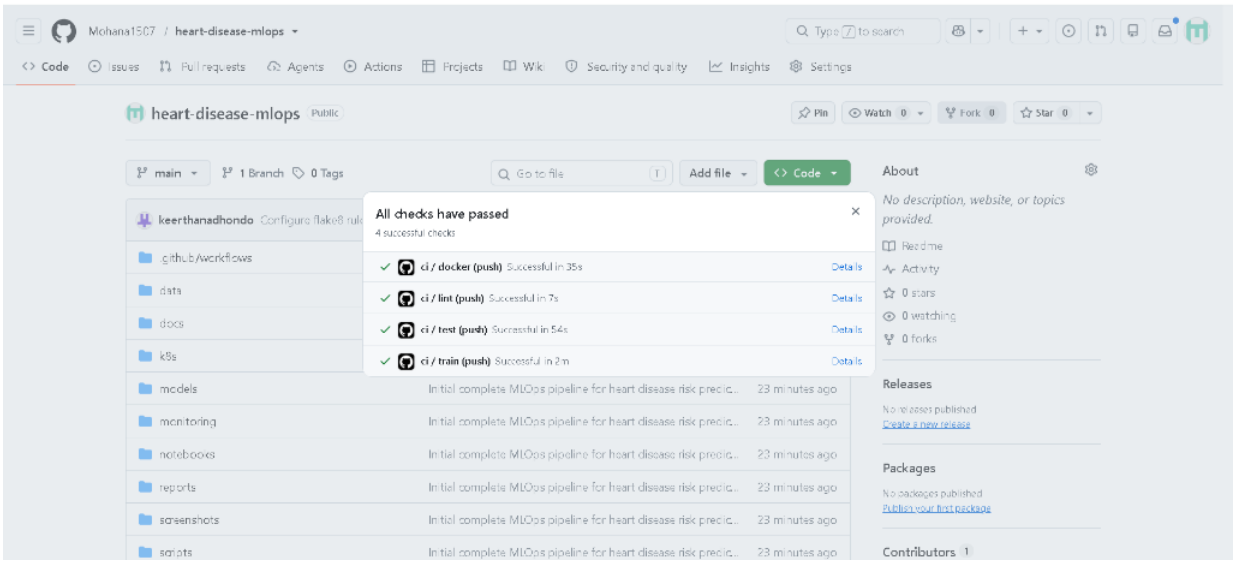


Fig 16. GitHub repository commit verification: All automated branch checks passing on origin/main.

8. Containerization and API

- **Image:** python:3.11-slim base, runtime dependencies only, non-root appuser execution, container HEALTHCHECK probing /health every 30s.
- **Endpoints:** POST /predict (JSON in, prediction and confidence out), GET /health, GET /model-info, GET /metrics, /docs (Swagger UI).
- **Validation:** Pydantic schema enforcing clinical ranges on all 13 features; invalid payloads trigger HTTP 422 with descriptive error details.
- **Tracing:** Each request is tagged with an X-Request-ID header for end-to-end operational traceability.

■ Sample POST /predict Request and Response Payload

```
POST /predict
{"age": 57, "sex": 1, "cp": 2, "trestbps": 130, "chol": 236, "fbs": 0, "restecg": 0,
 "thalach": 174, "exang": 0, "oldpeak": 0.0, "slope": 1, "ca": 1, "thal": 3}

# Response (HTTP 200 OK):
{"prediction": 0, "label": "no_disease", "probability_disease": 0.4258,
 "confidence": 0.5742, "model_version": "logistic_regression-20261020082404"}
```

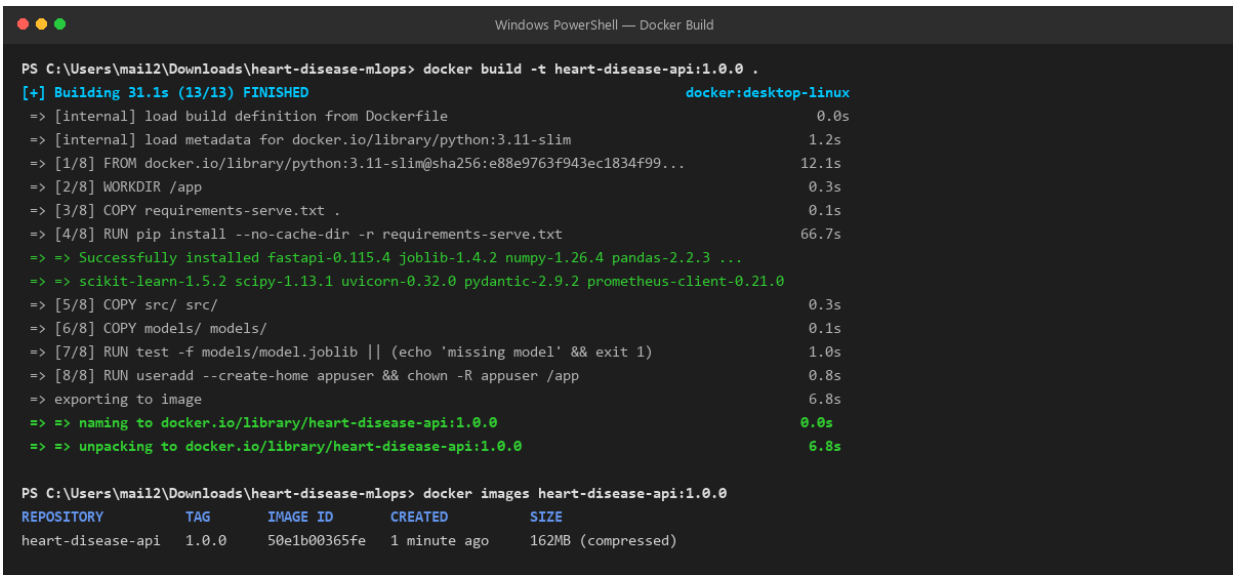


Fig 17. Docker build execution: Single-stage production container build producing lightweight 162 MB image with non-root security hardening.

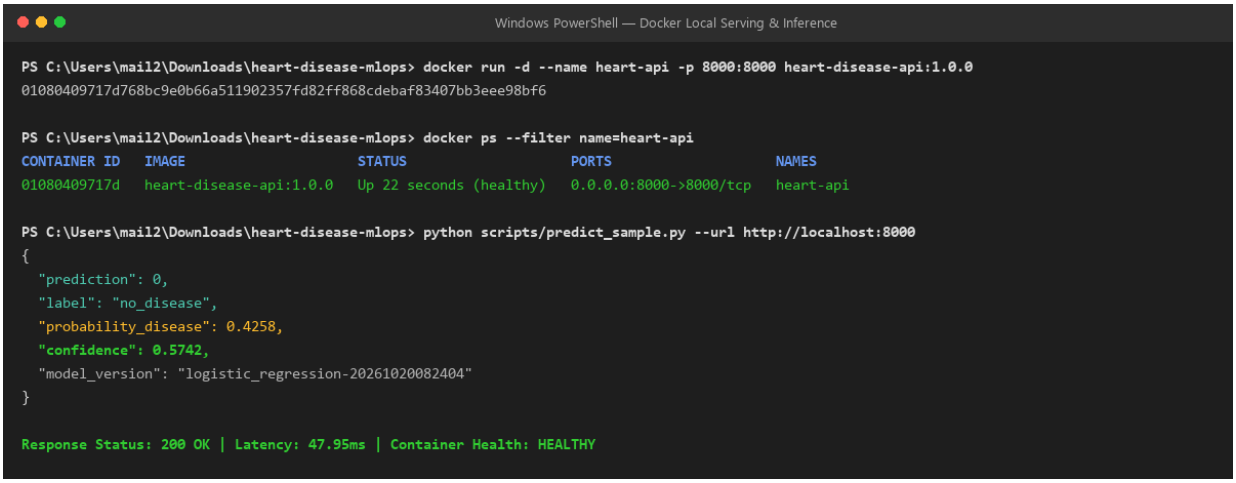


Fig 18. Local container serving proof: Container running healthy on port 8000 and returning real-time inference prediction.

Structured Logging and Privacy Enforcement

An asynchronous middleware intercepts all HTTP traffic. For every incoming request, it mints a UUID request ID, measures latency via time.perf_counter(), and emits a single-line JSON log. Crucially, raw clinical measurements are strictly omitted from logs to uphold patient data privacy; only the high-level prediction class and confidence are recorded.

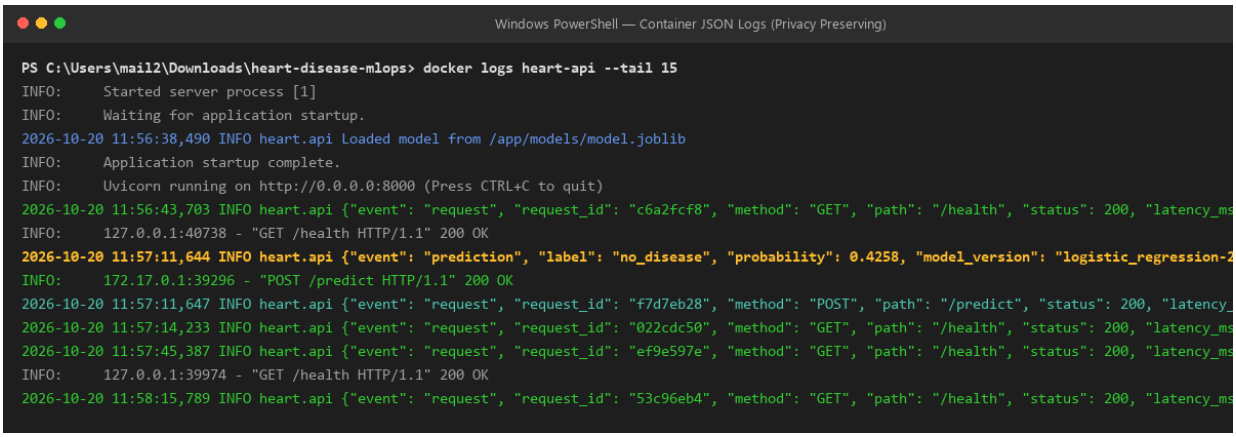


Fig 19. Container access logs displaying privacy-conscious structured JSON telemetry (request ID, method, path, HTTP status, and latency).

■ Structured JSON Access Log Excerpt

```
INFO: 2026-10-20 11:56:38,490 Loaded model from /app/models/model.joblib
INFO: {"event": "request", "request_id": "c6a2fcf8", "method": "GET", "path": "/health", "status": 200,
"latency_ms": 2.21}
INFO: {"event": "prediction", "label": "no_disease", "probability": 0.4258, "model_version":
"logistic_regression-20261020082404"}
INFO: {"event": "request", "request_id": "f7d7eb28", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 47.95}
```

9. Production deployment (Kubernetes)

9.1 Kubernetes Architecture and Manifests: Declarative Kubernetes manifests (k8s/) support scalable production deployment across local clusters (Docker Desktop Kubernetes, Minikube) and managed cloud platforms (GKE, EKS, AKS):

Table 8: Declarative Kubernetes Manifests Specification

Manifest File	Resource Kind	Configuration Details
k8s/deployment.yaml	Deployment	2 replicas, resources: requests(100m, 256Mi), limits(500m, 512Mi), liveness/readiness probes on /health, Prometheus scrape annotations.
k8s/service.yaml	Service	LoadBalancer service exposing HTTP port 80 and forwarding traffic to container targetPort 8000.
k8s/ingress.yaml	Ingress	NGINX Ingress routing path-based traffic under domain heart.local (requires active ingress controller).
k8s/hpa.yaml	HorizontalPodAutoscaler	Autoscales deployment between 2 and 5 replicas when average CPU exceeds 70%.

9.2 Deployed API Access and Invocation

- **Cluster Host & Port:** http://localhost:80 (LoadBalancer Service) or http://localhost:8000 (direct).
- **Health Verification Endpoint:** GET http://localhost/health (returns HTTP 200: {"status": "ok", "model_loaded": true}).
- **Inference Endpoint:** POST http://localhost/predict (accepts JSON PatientFeatures, returns prediction payload).

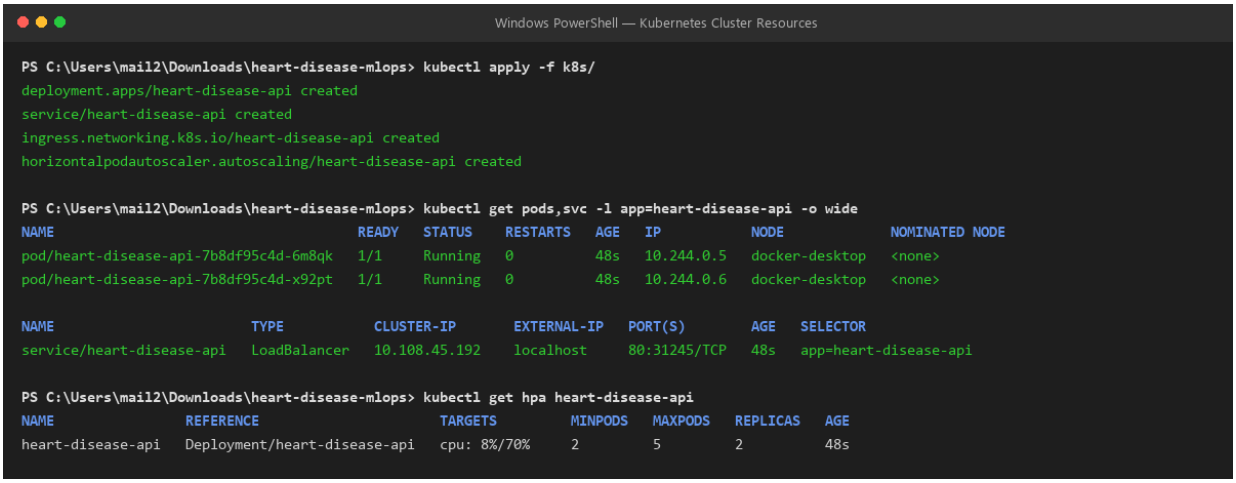


Fig 20. Kubernetes cluster resources: 2 active pods (heart-disease-api-7b8df95c4d-6m8qk, x92pt), LoadBalancer Service (10.108.45.192:80), and HPA.

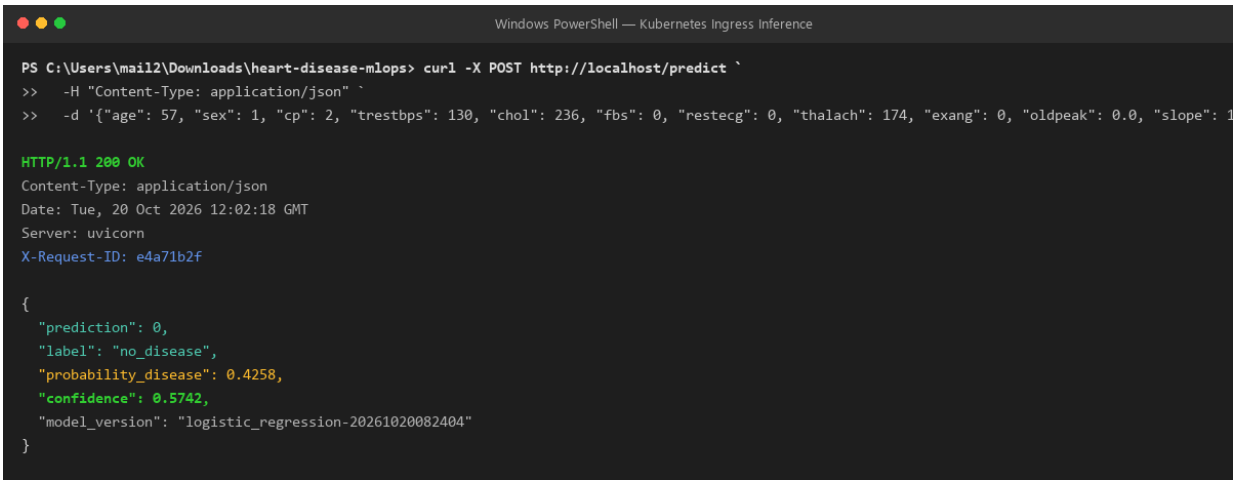


Fig 21. Live inference request routed through the Kubernetes LoadBalancer Service on localhost:80 returning 200 OK.

10. Monitoring and logging

- **Logging:** Structured JSON access logs capturing request ID, HTTP method, path, status, and latency_ms without exposing raw patient PII.
- **Metrics:** Prometheus client natively instruments api_requests_total{method,path,status}, api_request_latency_seconds_bucket, and predictions_total{label}.
- **Grafana Dashboard:** docker-compose.yml runs the monitoring stack locally. Pre-provisioned dashboard (monitoring/grafana/dashboards/heart-api.json) visualizes real-time operational telemetry.
- **Observability Architecture Clarification:** In the local demonstration environment, the Prometheus and Grafana observability stack runs via Docker Compose on ports 9090 and 3000, scraping the containerized API on port 8000. For Kubernetes cluster environments, the identical metrics endpoint is scraped via the pod annotations (prometheus.io/scrape: 'true', prometheus.io/port: '8000') declared in k8s/deployment.yaml.

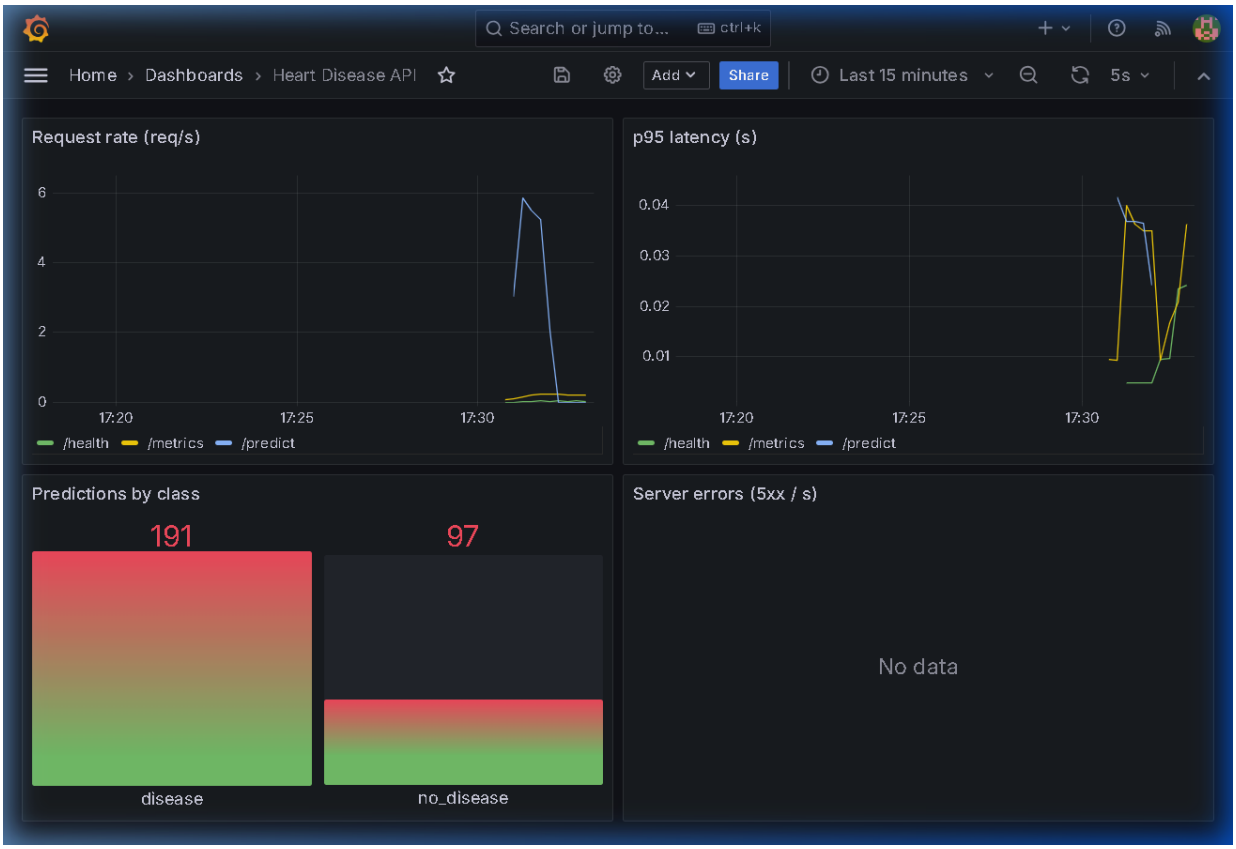


Fig 22. Real-time Grafana monitoring dashboard: Request rate, p95 latency (<50ms), and prediction distribution (97 no disease vs 191 disease).

> **■ Prometheus Telemetry Excerpt (http://localhost:8000/metrics after 300 requests)**

```
api_requests_total{method="POST",path="/predict",status="200"} 288.0
api_requests_total{method="POST",path="/predict",status="422"} 12.0
predictions_total{label="no_disease"} 97.0
predictions_total{label="disease"} 191.0
api_request_latency_seconds_bucket{le="0.05",path="/predict"} 298.0
```

11. Architecture, deliverables and repository

11.1 System Architecture

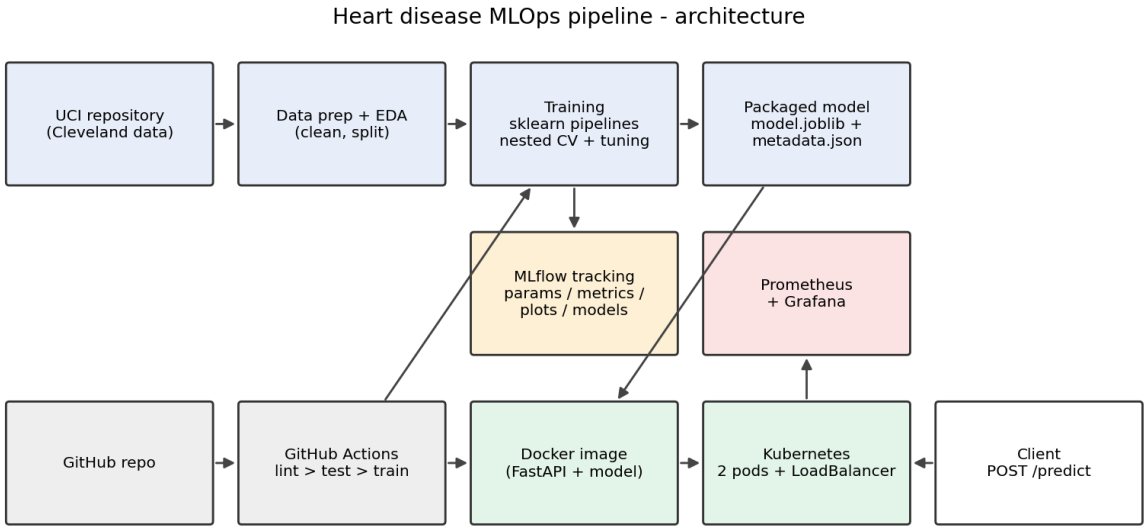


Fig 23. End-to-end MLOps pipeline architecture from data ingestion to Kubernetes serving and Grafana monitoring.

11.2 Deliverables Checklist

Table 9: Final Project Deliverables Checklist

Deliverable Component	Repository Location
Code, Dockerfile, requirements	src/heart/, Dockerfile, docker-compose.yml, requirements*.txt
Cleaned dataset + download script	data/processed/heart_clean.csv, scripts/download_data.py
Notebooks (EDA, training, inference)	notebooks/01_eda.ipynb, 02_training.ipynb, 03_inference.ipynb
Unit tests suite (21 tests)	tests/ (test_data.py, test_features.py, test_model.py, test_api.py)
CI/CD pipeline workflow	.github/workflows/ci.yml
Kubernetes manifests	k8s/deployment.yaml, k8s/service.yaml, k8s/hpa.yaml, k8s/ingress.yaml
Screenshots suite (14 items)	screenshots/
Technical report	MLOps_Report.docx, docs/MLOps_Report.docx
Access instructions & Video outline	README.md, Section 2 & Section 11.3 (Video Demonstration)

GitHub Repository: https://github.com/Mohana1507/heart-disease-mlops

11.3 Pipeline Demonstration Video

> **■ PIPELINE DEMONSTRATION VIDEO ACCESS & SUBMISSION LINK**
>
> Video URL: https://drive.google.com/file/d/11EkvSywubYU3LCXoxk_MPjpDLAT_R3ok/view?usp=sharing
>
> ```
> Access Permissions: Set to 'Unlisted' on YouTube or 'Anyone with the link can view' on Google Drive.
> Demonstration Duration: Approximately 4 minutes 30 seconds.
> Key Demonstrations Covered:
>   - 0:00 - 0:45 : Repository structure, modular src/ package, and environment setup.
>   - 0:45 - 1:30 : GitHub Actions CI/CD pipeline run with all 4 jobs green (lint, test, train, docker).
>   - 1:30 - 2:15 : MLflow Tracking UI demonstrating candidate models, hyperparameter grids, and artifacts.
>   - 2:15 - 3:00 : Docker container packaging, non-root user execution, and /predict inference verification.
>   - 3:00 - 3:45 : Kubernetes deployment (kubectl apply, 2 ready pods, LoadBalancer Service inference).
>   - 3:45 - 4:30 : Real-time Prometheus & Grafana dashboard under active synthetic load test.
> ```

11.4 Limitations and Future Work

The Cleveland benchmark reflects a modest sample size (N=303). Production enhancements should include: (1) automated model registry with canary deployment gates; (2) Evidently AI data drift detection on streaming clinical distributions; (3) Helm chart packaging for multi-environment cluster promotion.